

PROJECT SANCTUM

Financial Intelligence Layer

Feature Extension

BRD / PRD / Technical Specification

Cyrill Adrian Wicaksono | u7962766

Australian National University | COMP3740

Version 1.0 | April 2026

Table of Contents

Table of Contents	2
1.1 Overview	4
1.2 Problem Statement	4
1.3 Goals and Objectives	4
1.4 Key Assumptions	5
1.5 Impact	5
1.6 Scope	5
In Scope	5
Out of Scope	6
1.7 User Roles	6
1.8 Third-Party Dependencies	6
1.9 Timeline	7
2.1 Requirements Summary	8
2.2 Epic 1: BillReminder Model Extension	8
2.2.1 User Stories and Acceptance Criteria	8
2.3 Epic 2: Financial Intelligence Engine	9
2.3.1 User Stories and Acceptance Criteria	9
2.4 Epic 3: Financial Health Score Widget	10
2.4.1 User Stories and Acceptance Criteria	10
2.5 Epic 4: Insights Feed Widget	10
2.5.1 User Stories and Acceptance Criteria	11
2.6 Non-Functional Requirements	11
3.1 System Architecture Overview	12
3.2 Tech Stack	12
3.3 Data Model Changes	13
3.3.1 BillReminder (Extended)	13
3.3.2 New Turtle Predicates	13
3.3.3 FinancialIntelligenceResult	14
3.3.4 InsightString	14
3.4 Financial Intelligence Engine: Complete Rule Specification	14
PILLAR 1: Budget Adherence Score (weight 40%)	15
Rule BA-01: Over-Budget Category Penalty	15
Rule BA-02: Budget Utilisation Proximity Penalty	15
Rule BA-03: No-Spend Category Bonus	15
Rule BA-04: Consistent Under-Budget Performance	16
Rule BA-05: Unbudgeted Spending Penalty	16
Rule BA-06: Budget Coverage Bonus	16
PILLAR 2: Bill Reliability Score (weight 35%)	17
Rule BR-01: Overdue Unpaid Bill Penalty	17

Rule BR-02: Late Payment Pattern Penalty	17
Rule BR-03: On-Time Payment Streak Bonus	18
Rule BR-04: Notification Responsiveness Bonus	18
Rule BR-05: Recurring Bill Reliability Bonus	18
Rule BR-06: No Bills Registered Neutral Baseline	19
PILLAR 3: Spending Consistency Score (weight 25%)	19
Rule SC-01: Month-Over-Month Spend Spike Penalty	19
Rule SC-02: Category Spend Drift Penalty	19
Rule SC-03: Spending Frequency Consistency	20
Rule SC-04: Single-Category Concentration Penalty	20
Rule SC-05: Positive Trend Bonus	20
Rule SC-06: No Transaction Data Neutral Baseline	21
Insight Templates	21
Template IT-01: Over-Budget Alert (severity: alert, category: budget)	21
Template IT-02: Approaching Budget Warning (severity: warning, category: budget)	21
Template IT-03: Overdue Bill Alert (severity: alert, category: bills)	21
Template IT-04: Upcoming Bill Warning (severity: warning, category: bills)	22
Template IT-05: Spending Spike Warning (severity: warning, category: spending)	22
Template IT-06: Category Drift Warning (severity: warning, category: spending)	22
Template IT-07: Positive Trend Celebration (severity: info, category: spending)	22
Template IT-08: On-Time Payment Streak Celebration (severity: info, category: bills)	22
Template IT-09: Consistent Under-Budget Celebration (severity: info, category: budget)	23
Template IT-10: Unbudgeted Spending Nudge (severity: warning, category: budget)	23
Template IT-11: Single-Category Concentration Warning (severity: warning, category: spending)	23
Template IT-12: General Health Encouragement (severity: info, category: general)	23
3.5 Riverpod Provider Design	23
financialIntelligenceProvider	24
3.6 API Endpoint Specifications	24
3.7 Authentication and Authorization	24
3.8 Key Technical Decisions and Trade-offs	24
A. Document Revision History	26
B. Sign-Offs	26
C. Supporting Documents and Links	26

PART 1: BUSINESS REQUIREMENTS

1.1 Overview

Project Sanctum is a Flutter-based personal finance application built on SOLID Pods, giving users complete data sovereignty over their financial records. All user data is stored encrypted in the user's own SOLID Pod on solidcommunity.au, no data ever touches a centralised application server.

This document specifies the Financial Intelligence Layer, a Feature Extension feature extension that adds on-device rule-based analysis, a Financial Health Score, and a natural-language Insights Feed to the existing Dashboard. The feature deepens the existing application without introducing new data sources, external API calls, or architectural changes.

1.2 Problem Statement

The current Sanctum dashboard presents raw financial data, transaction lists, budget progress bars, and bill reminders, but provides no interpretation. Users must mentally compute whether they are financially healthy, which categories are problematic, and whether their bill payment behaviour is reliable. This cognitive gap reduces the perceived value of the application and limits its "wow factor" for both end users and academic assessors.

The specific problems this feature solves:

- Users cannot tell at a glance whether their overall financial situation is improving or worsening.
- Budget progress bars show individual category status but give no holistic picture.
- Bill reminders track due dates but provide no insight into payment reliability patterns.
- Spending trends across months are invisible, the app shows current state, not trajectory.
- The BillReminder model has no field recording when a notification was dispatched, making it impossible to audit the notification lifecycle or use notification timing in health computation.

1.3 Goals and Objectives

Goal	Metric	Target
Provide a holistic financial health signal	Financial Health Score visible on dashboard	100% of authenticated sessions show a score
Surface actionable, friendly insights	Insight strings generated per session	3 to 5 insights per dashboard load

Goal	Metric	Target
Enrich the BillReminder lifecycle model	New fields present in Pod Turtle files	notificationDate and paidDate persisted correctly
Maintain full data sovereignty	External network calls from the engine	Zero, all computation is on-device
Build within sprint budget	Developer hours consumed	Under 20 hours including tests

1.4 Key Assumptions

- Sprints 1 through 6 are complete and all three Riverpod providers (transactionListProvider, budgetListProvider, billReminderListProvider) load real data from the SOLID Pod.
- The Flutter app targets web (primary demo platform) with Android and iOS as secondary targets.
- No minimum data threshold is enforced, the engine produces meaningful output even from a single transaction, gracefully degrading insight depth as data increases.
- The assessor and demo audience understand that "rule-based intelligence" is a deliberate architectural choice aligned with SOLID data sovereignty, not a limitation.
- The existing rdflib Turtle serialization layer in PodService is stable and can be extended with two new fields without breaking existing Pod files.

1.5 Impact

For the demo audience: the dashboard transforms from a data display into an intelligent financial advisor that speaks to the user in plain language, visually anchored by a scored health gauge. This is the "wow factor"; intelligence without sacrificing privacy.

For the ANU assessor: the feature demonstrates mastery of derived state management (Riverpod computed providers), clean separation between data and analysis layers, and principled architectural decision-making aligned with the project's data sovereignty thesis.

For the SOLID ecosystem: the feature proves that meaningful financial intelligence does not require centralised ML infrastructure, it can be computed locally from user-owned data, preserving the trust model that SOLID is built on.

1.6 Scope

In Scope

- Financial Health Score widget on the Dashboard screen (score 0–100, three visible sub-scores, colour-coded grade label).

- Insights Feed widget on the Dashboard screen (3–5 natural-language strings, friendly coach tone, rule-based generation).
- BillReminder data model extension: two new fields (notificationDate: DateTime? and paidDate: DateTime?).
- PodService Turtle serializer and parser updates for the two new BillReminder fields.
- FinancialIntelligenceService: a pure-Dart class implementing the full rule-based engine.
- financialIntelligenceProvider: a Riverpod derived AsyncNotifier composing all three existing providers.
- Full rule-based engine covering 18 named rules across three scoring pillars and 12 insight templates.
- Bill notification scheduling update: automatically write notificationDate to Pod when a local notification is dispatched.
- Feature Extension task entries in the Notion progress tracker.

Out of Scope

- Any external API call, cloud ML inference, or on-device model loading.
- A dedicated Insights tab, all new UI lives within the existing Dashboard screen.
- Historical trend computation beyond what is already stored in the Pod (no new data collection).
- Permissioning or sharing of the health score with third parties.
- Push notifications (existing local notifications are used; no new notification channels).

1.7 User Roles

Role	Description	Interaction with This Feature
Authenticated User	Has logged in via SolidLogin and has an active SOLID Pod session	Sees Financial Health Score and Insights Feed on every Dashboard load
ANU Assessor	Evaluates the project for COMP8715	Views demo or reads report; cares about architecture and sovereignty story
Demo Audience	Observers watching a live demonstration	Sees the dashboard widget and hears the coach-tone insights read aloud

1.8 Third-Party Dependencies

Dependency	Purpose	Sovereignty Impact
flutter_riverpod (existing)	Derived provider composition	None — local only
flutter_local_notifications (existing)	Notification dispatch; notificationDate written to Pod on send	None — local only
rdflib (existing)	Turtle serialization of new BillReminder fields	None — local only
solidpod (existing)	writePod for updated BillReminder Turtle files	Data stays in user Pod
No new dependencies	The engine is pure Dart	Zero external calls

1.9 Timeline

Week	Focus	Key Deliverables
Week 1, Days 1–2 (~6 hrs)	Data model + PodService extension	BillReminder updated, Turtle serializer updated, existing tests pass
Week 1, Days 3–5 (~7 hrs)	FinancialIntelligenceService + provider	Full rule engine implemented, unit tests for all 18 rules and 12 templates
Week 2, Days 1–3 (~5 hrs)	Dashboard UI widgets	Health Score widget and Insights Feed widget rendering on Dashboard
Week 2, Days 4–5 (~2 hrs)	Polish, edge cases, Notion log	Empty-state handling, colour coding, Feature Extension Notion tasks marked Done

PART 2: PRODUCT REQUIREMENTS

2.1 Requirements Summary

ID	Epic	Priority	Effort (hrs)
E1	BillReminder Model Extension	Must Have	3
E2	Financial Intelligence Engine	Must Have	8
E3	Financial Health Score Widget	Must Have	5
E4	Insights Feed Widget	Must Have	4

2.2 Epic 1: BillReminder Model Extension

Extend the BillReminder Dart class and its Turtle representation with two new optional fields that enable the Financial Intelligence Engine to compute bill reliability scores and generate payment-timing insights.

2.2.1 User Stories and Acceptance Criteria

Story: *As a user, I want the app to automatically record when a bill notification was sent to me, so that I can trust that my notification history is accurate and auditable.*

Acceptance Criteria:

- When flutter_local_notifications dispatches a bill notification, the app writes the current DateTime to notificationDate on the corresponding BillReminder and calls PodService.updateBillReminder() to persist it to the Pod.
- The notificationDate field is optional (DateTime?), BillReminder objects created before this sprint have null notificationDate and the app handles this gracefully.
- The Turtle representation includes fin:notificationDate as an xsd:dateTime literal when the field is non-null, and omits the predicate entirely when null.
- The Turtle parser reads fin:notificationDate and populates the field; if the predicate is absent, the field is null.
- Existing BillReminder Pod files without fin:notificationDate parse without error.

Story: *As a user, I want the app to record the exact date I paid a bill, so that I can see my payment reliability history and the app can tell me whether I tend to pay on time.*

Acceptance Criteria:

- When the user taps "Mark as Paid", the app writes the current DateTime to paidDate on the BillReminder before calling PodService.updateBillReminder().
- The paidDate field is optional (DateTime?), it is null until the user marks the bill as paid.

- The Turtle representation includes `fin:paidDate` as an `xsd:dateTime` literal when non-null.
- The days-late calculation is: `paidDate.difference(dueDate).inDays`, positive means late, negative or zero means on time.
- The `BillReminder` class's existing `isPaid` field is kept; `paidDate` is additive and does not replace it.

2.3 Epic 2: Financial Intelligence Engine

A pure-Dart `FinancialIntelligenceService` class that takes Lists of `Transaction`, `Budget`, and `BillReminder` objects and produces a `FinancialIntelligenceResult` containing a scored `HealthScore` and a List of `InsightString`. The engine contains 18 named rules across three scoring pillars and 12 insight template functions.

2.3.1 User Stories and Acceptance Criteria

Story: *As a developer, I want the financial intelligence engine to be a stateless pure-Dart service that accepts plain Dart objects and returns a result object, so that it is trivially testable without mocking Pod calls.*

Acceptance Criteria:

- `FinancialIntelligenceService` is instantiated with no constructor parameters.
- The single public method is: `FinancialIntelligenceResult analyse(List<Transaction> txs, List<Budget> budgets, List<BillReminder> bills)`.
- The method is synchronous — no `async`, no `Future`, no I/O.
- Unit tests cover all 18 rules individually using in-memory fixture data.
- The service imports nothing from `solidpod`, `rdflib`, or `flutter` packages, pure Dart only.

Story: *As a user, I want the Financial Health Score to reflect three dimensions of my financial behaviour, so that I understand what is driving my overall score.*

Acceptance Criteria:

- `FinancialIntelligenceResult` exposes: `int overallScore` (0–100), `int budgetAdherenceScore` (0–100), `int billReliabilityScore` (0–100), `int spendingConsistencyScore` (0–100), `String grade` ("Excellent", "Good", "Needs Attention", "At Risk").
- $\text{overallScore} = (\text{budgetAdherenceScore} * 0.40) + (\text{billReliabilityScore} * 0.35) + (\text{spendingConsistencyScore} * 0.25)$, rounded to nearest integer.
- Grade thresholds: 80–100 = "Excellent", 60–79 = "Good", 40–59 = "Needs Attention", 0–39 = "At Risk".
- Each sub-score is independently computed by its pillar rules and clamped to 0–100.
- When all input lists are empty, all scores are 50 (neutral baseline) and grade is "Good".

Story: *As a user, I want to receive 3 to 5 friendly, specific, actionable insights on my dashboard, so that I know what to do next with my finances.*

Acceptance Criteria:

- FinancialIntelligenceResult exposes List<InsightString> insights, where InsightString has: String text, InsightSeverity severity (info, warning, alert), InsightCategory category (budget, bills, spending, general).
- The engine selects the 3 most severe and relevant insights from all triggered templates, with a maximum of 5.
- Insights are ordered: alerts first, then warnings, then info.
- Each insight string is 1–2 sentences in friendly coach tone, uses the user's real category names, amounts (formatted as currency), and dates where applicable.
- No insight is generated for a pillar that has zero data, the template is skipped and a lower-priority insight from another pillar fills the slot.

2.4 Epic 3: Financial Health Score Widget

A Flutter widget rendered at the top of the Dashboard screen showing the overall score, grade label, and three sub-score breakdown bars. The widget reads from financialIntelligenceProvider.

2.4.1 User Stories and Acceptance Criteria

Story: *As a user, I want to see my Financial Health Score prominently at the top of my dashboard as a large number with a colour-coded grade, so that I can immediately understand my financial health at a glance.*

Acceptance Criteria:

- The widget displays overallScore as a large numeral (48sp or above) with the grade label directly below it.
- Colour coding: Excellent = green (#4CAF50), Good = blue (#1F4788), Needs Attention = amber (#FF9800), At Risk = red (#F44336).
- The three sub-score labels are shown as labelled linear progress bars below the main score: "Budget Adherence", "Bill Reliability", "Spending Consistency".
- Each sub-score bar is filled proportionally (value / 100) and uses the same colour coding thresholds applied per sub-score.
- While financialIntelligenceProvider is loading, a shimmer placeholder occupies the widget space.
- If the provider returns an error, a soft error card is shown: "Could not compute insights. Pull to refresh."

2.5 Epic 4: Insights Feed Widget

A Flutter widget rendered below the Health Score widget on the Dashboard screen, displaying the ordered list of insight strings with severity icons and category badges.

2.5.1 User Stories and Acceptance Criteria

Story: *As a user, I want to see a feed of specific, friendly financial insights below my health score, so that I know exactly what actions to take to improve my financial situation.*

Acceptance Criteria:

- The widget renders each InsightString as a card with: a left-border colour (alert = red, warning = amber, info = blue), a severity icon, a category badge (Budget / Bills / Spending / General), and the insight text.
- Cards are ordered alerts first, then warnings, then info.
- A maximum of 5 insight cards are shown; a minimum of 3 are always shown (general fallback insights fill slots if fewer than 3 specific insights fire).
- Each card is tappable: tapping navigates to the relevant screen (budget insight -> Budgets tab, bills insight -> Bills tab, spending insight -> Transactions tab).
- While loading, each card slot shows a shimmer placeholder row.
- An empty state is never shown: at minimum the three general fallback insights always render.

2.6 Non-Functional Requirements

Category	Requirement	Target
Performance	Time to compute FinancialIntelligenceResult after provider data is loaded	Under 50ms on a mid-range device (pure Dart, no I/O)
Data Sovereignty	External network calls made by the intelligence engine	Zero — all computation is local
Reliability	App crash rate attributable to the new feature	Zero — all engine calls wrapped in try/catch; errors surface as UI state, not exceptions
Backwards Compatibility	Behaviour when loading BillReminder Pod files written before Feature Extension	Fully backwards compatible — missing fields parsed as null without error
Testability	Unit test coverage of FinancialIntelligenceService rules	100% of named rules have at least one unit test
Accessibility	Score widget colour reliance	Grade label text always accompanies colour — colour is not the sole signal

PART 3: TECHNICAL IMPLEMENTATION

3.1 System Architecture Overview

The Financial Intelligence Layer is a pure computation layer that sits above the existing data providers and below the Dashboard UI. It introduces no new external dependencies, no new Pod file types, and no new navigation routes.

Data flow:

- `transactionListProvider`, `budgetListProvider`, `billReminderListProvider` (existing, Sprint 3–5) load data from SOLID Pod via `PodService`.
- `financialIntelligenceProvider` (new, Feature Extension) is a Riverpod derived `AsyncNotifier` that watches all three providers, extracts their data when all are in `AsyncData` state, and calls `FinancialIntelligenceService.analyse()` synchronously.
- `FinancialIntelligenceService` (new, Feature Extension) is a pure-Dart class with no constructor dependencies. It takes the three lists, runs all rules, and returns a `FinancialIntelligenceResult`.
- `HealthScoreWidget` and `InsightsWidget` (new, Feature Extension) are `ConsumerWidgets` that watch `financialIntelligenceProvider` and render the result.
- `BillReminder` model and `PodService` (extended, Feature Extension) gain two new nullable fields: `notificationDate` and `paidDate`.

Layered architecture (top to bottom):

- Dashboard Screen (UI layer)
- `HealthScoreWidget`, `InsightsWidget` (new presentation widgets)
- `financialIntelligenceProvider` (new derived Riverpod provider)
- `FinancialIntelligenceService` (new pure-Dart engine)
- `transactionListProvider`, `budgetListProvider`, `billReminderListProvider` (existing)
- `PodService` (extended with two new `BillReminder` fields)
- `solidpod`: `readPod`, `writePod`, `getDirUrl`, `getResourcesInContainer` (unchanged)
- SOLID Pod on `solidcommunity.au` (unchanged)

3.2 Tech Stack

Component	Technology	Version / Notes
UI Framework	Flutter	Existing — no change
State Management	<code>flutter_riverpod</code> <code>AsyncNotifier</code>	Existing pattern — new derived provider added

Component	Technology	Version / Notes
Intelligence Engine	Pure Dart (no packages)	New — zero external dependencies
Data Persistence	solidpod writePod / readPod	Existing — extended for two new fields
RDF Serialization	rdflib Dart package	Existing — two new predicates added
Notifications	flutter_local_notifications	Existing — notificationDate written on dispatch
Testing	flutter_test (dart:test)	Existing test harness — new unit tests added

3.3 Data Model Changes

3.3.1 BillReminder (Extended)

Two new nullable fields are added to the existing immutable class:

Field	Type	Nullable	Description
id	String	No	UUID — unchanged
name	String	No	Bill name — unchanged
amount	double	No	Bill amount — unchanged
dueDate	DateTime	No	Payment due date — unchanged
recurrence	String	No	"one-off" or "monthly" — unchanged
isPaid	bool	No	Payment status flag — unchanged
notificationDate	DateTime?	Yes	NEW: DateTime the local notification was dispatched. Null until notification fires. Written automatically by notification scheduling code.
paidDate	DateTime?	Yes	NEW: DateTime the user tapped "Mark as Paid". Null until paid. Enables days-late computation: <code>paidDate.difference(dueDate).inDays</code> .

3.3.2 New Turtle Predicates

Turtle namespace: @prefix fin: <http://sanctum.app/finance#>

Predicate	XSD Type	Serialization Rule
fin:notificationDate	xsd:dateTime	Omit predicate if null. Write as ISO 8601 string with <code>toISOString()</code> when non-null.
fin:paidDate	xsd:dateTime	Omit predicate if null. Write as ISO 8601 string with <code>toISOString()</code> when non-null.

3.3.3 FinancialIntelligenceResult

Field	Type	Description
overallScore	int	Weighted composite: $\text{budgetAdherence} \times 0.40 + \text{billReliability} \times 0.35 + \text{spendingConsistency} \times 0.25$. Clamped 0–100.
budgetAdherenceScore	int	Score 0–100 from Budget Adherence pillar rules. Clamped.
billReliabilityScore	int	Score 0–100 from Bill Reliability pillar rules. Clamped.
spendingConsistencyScore	int	Score 0–100 from Spending Consistency pillar rules. Clamped.
grade	String	"Excellent" (80–100), "Good" (60–79), "Needs Attention" (40–59), "At Risk" (0–39).
insights	List<InsightString>	Ordered list of 3–5 insight objects. Alerts first, then warnings, then info.

3.3.4 InsightString

Field	Type	Values
text	String	Natural language insight in friendly coach tone, 1–2 sentences.
severity	InsightSeverity (enum)	alert, warning, info
category	InsightCategory (enum)	budget, bills, spending, general

3.4 Financial Intelligence Engine: Complete Rule Specification

This section defines all 18 rules and 12 insight templates implemented in `FinancialIntelligenceService`. Rules are deterministic and stateless. All monetary values are

formatted with the intl NumberFormat.currency locale. All percentages are rounded to the nearest integer.

PILLAR 1: Budget Adherence Score (weight 40%)

Base score: 100. Rules apply deductions and bonuses. Final score clamped to 0–100. Computed over the current calendar month.

Rule BA-01: Over-Budget Category Penalty

Attribute	Value
Name	BA-01 Over-Budget Category Penalty
Input	List<Budget> budgets (current month), List<Transaction> txs (current month)
Logic	For each budget where $\text{spentAmount} > \text{monthlyLimit}$: deduct 15 points per over-budget category. $\text{spentAmount} = \text{sum of tx.amount where tx.category} == \text{budget.category and tx.date is in budget.month}$.
Maximum deduction	60 points (4 categories over budget = floor)
Edge case	If no budgets exist, rule is skipped (score unchanged at 100 for this rule).

Rule BA-02: Budget Utilisation Proximity Penalty

Attribute	Value
Name	BA-02 Budget Utilisation Proximity Penalty
Input	Budgets and transactions as per BA-01
Logic	For each budget where $90\% \leq \text{spentAmount}/\text{monthlyLimit} < 100\%$: deduct 5 points per such category. This warns that the user is approaching but has not yet breached limits.
Maximum deduction	20 points
Rationale	Distinguishes "over budget" (BA-01) from "dangerously close" (BA-02) to provide nuanced scoring.

Rule BA-03: No-Spend Category Bonus

Attribute	Value
Name	BA-03 No-Spend Category Bonus

Attribute	Value
Input	Budgets and transactions as per BA-01
Logic	For each budget where spentAmount == 0 (user has a budget set but spent nothing in that category this month): add 5 points per such category.
Maximum bonus	15 points (cap at 3 categories to avoid inflation)
Edge case	Only applies to categories that have an active budget — unbudgeted zero-spend categories are ignored.

Rule BA-04: Consistent Under-Budget Performance

Attribute	Value
Name	BA-04 Consistent Under-Budget Performance
Input	All transactions grouped by month, all budgets
Logic	For each budget category: if spentAmount < monthlyLimit for 3 or more distinct months available in transaction history: add 10 points per such category.
Maximum bonus	20 points (cap at 2 categories)
Minimum data	Requires at least 2 months of transaction data. Skipped if fewer months available.

Rule BA-05: Unbudgeted Spending Penalty

Attribute	Value
Name	BA-05 Unbudgeted Spending Penalty
Input	Current month transactions, current month budgets
Logic	Identify transaction categories that have no corresponding budget entry. If total unbudgeted spending > 20% of total spending this month: deduct 10 points. If > 40%: deduct 20 points.
Rationale	Encourages the user to set budgets for all meaningful spending categories.
Edge case	If no budgets exist at all, this rule is skipped (cannot penalise absence of budgets when user has not set any).

Rule BA-06: Budget Coverage Bonus

Attribute	Value
Name	BA-06 Budget Coverage Bonus
Input	Current month transactions, current month budgets
Logic	Calculate: $\text{budgetedCategories} / \text{totalUniqueTransactionCategories}$. If $\geq 80\%$: add 10 points. If $\geq 60\%$: add 5 points.
Rationale	Rewards users who have budgeted comprehensively across their spending categories.

PILLAR 2: Bill Reliability Score (weight 35%)

Base score: 100. Rules apply deductions. Computed over all BillReminder records (not month-scoped — bill history accumulates over time).

Rule BR-01: Overdue Unpaid Bill Penalty

Attribute	Value
Name	BR-01 Overdue Unpaid Bill Penalty
Input	List<BillReminder> bills
Logic	For each bill where $\text{isPaid} == \text{false}$ and $\text{dueDate} < \text{DateTime.now}()$: deduct 20 points per overdue unpaid bill. This is the most severe bill penalty — an overdue bill is an active financial risk.
Maximum deduction	60 points (3 overdue bills = floor for this pillar)

Rule BR-02: Late Payment Pattern Penalty

Attribute	Value
Name	BR-02 Late Payment Pattern Penalty
Input	Bills where $\text{isPaid} == \text{true}$ and $\text{paidDate} != \text{null}$
Logic	$\text{daysLate} = \text{paidDate.difference}(\text{dueDate}).\text{inDays}$. For each bill where $\text{daysLate} > 0$: deduct 8 points per late payment. For each bill where $\text{daysLate} > 7$: deduct an additional 8 points (stacking — very late = 16 points total).
Maximum deduction	40 points

Attribute	Value
Edge case	If paidDate is null (bill paid before Feature Extension model extension), this rule is skipped for that record — no retrospective penalty for missing data.

Rule BR-03: On-Time Payment Streak Bonus

Attribute	Value
Name	BR-03 On-Time Payment Streak Bonus
Input	Bills where isPaid == true and paidDate != null
Logic	Count consecutive bills (sorted by dueDate descending) where daysLate <= 0. If streak >= 3: add 10 points. If streak >= 5: add 20 points (replaces, not stacks with, the 3-bill bonus).
Maximum bonus	20 points
Rationale	Rewards sustained reliable payment behaviour, not just a single on-time payment.

Rule BR-04: Notification Responsiveness Bonus

Attribute	Value
Name	BR-04 Notification Responsiveness Bonus
Input	Bills where notificationDate != null and isPaid == true and paidDate != null
Logic	daysFromNotificationToPaid = paidDate.difference(notificationDate).inDays. If the user paid within 2 days of receiving the notification: add 5 points per such bill, up to a maximum of 10 points. This rewards users who act on reminders promptly.
Edge case	Skipped entirely if notificationDate is null — no penalty for absence of notification data.

Rule BR-05: Recurring Bill Reliability Bonus

Attribute	Value
Name	BR-05 Recurring Bill Reliability Bonus
Input	Bills where recurrence == "monthly"
Logic	Count monthly recurring bills that are: either not yet due (dueDate >= DateTime.now()) or already paid (isPaid == true). If all monthly bills fall into these states (none overdue and unpaid): add 10 points.

Attribute	Value
Rationale	Specifically rewards users who manage their recurring obligations reliably.

Rule BR-06: No Bills Registered Neutral Baseline

Attribute	Value
Name	BR-06 No Bills Registered Neutral Baseline
Input	List<BillReminder> bills (empty)
Logic	If the bills list is empty: set billReliabilityScore to 50 (neutral baseline) regardless of other rules. No penalty for not using the bills feature.
Rationale	A user who has not set up bill reminders should not be penalised — the score should reflect behaviour, not feature adoption.

PILLAR 3: Spending Consistency Score (weight 25%)

Base score: 100. Computed by comparing month-over-month spending patterns. Requires at least 2 months of transaction data. If fewer than 2 months exist, score defaults to 60 (slightly-below-neutral, data-sparse baseline).

Rule SC-01: Month-Over-Month Spend Spike Penalty

Attribute	Value
Name	SC-01 Month-Over-Month Spend Spike Penalty
Input	All transactions grouped by calendar month
Logic	Compare currentMonthTotal vs previousMonthTotal. If currentMonthTotal > previousMonthTotal * 1.30 (30% spike): deduct 15 points. If > 1.50 (50% spike): deduct 30 points (replaces, not stacks).
Edge case	Skipped if fewer than 2 months of data.

Rule SC-02: Category Spend Drift Penalty

Attribute	Value
Name	SC-02 Category Spend Drift Penalty
Input	Transactions grouped by category and month
Logic	For each category that appears in both the current and previous month: calculate the percentage change in spend. For each category where spend

Attribute	Value
	increased by more than 50%: deduct 5 points. Maximum deduction: 20 points across all categories.
Rationale	Penalises category-level volatility even when the overall total is stable — a sign of erratic spending habits in specific areas.

Rule SC-03: Spending Frequency Consistency

Attribute	Value
Name	SC-03 Spending Frequency Consistency
Input	Transactions grouped by calendar month
Logic	Calculate transaction count per month. If the current month's transaction count is within 30% of the previous month's count (i.e. neither significantly more nor fewer transactions): add 10 points.
Rationale	Consistent transaction frequency suggests stable spending habits — not just similar totals but similar behaviour.

Rule SC-04: Single-Category Concentration Penalty

Attribute	Value
Name	SC-04 Single-Category Concentration Penalty
Input	Current month transactions
Logic	If one single category accounts for more than 60% of total spending: deduct 10 points. If more than 80%: deduct 20 points (replaces). This flags over-reliance on one spending area.
Edge case	Skipped if fewer than 3 transactions this month (insufficient data for meaningful concentration analysis).

Rule SC-05: Positive Trend Bonus

Attribute	Value
Name	SC-05 Positive Trend Bonus
Input	Transactions grouped by calendar month (requires 3+ months)
Logic	If total spending decreased month-over-month for 2 consecutive months (month3 < month2 < month1 in descending chronological order): add 15 points.

Attribute	Value
Rationale	Explicitly rewards a downward spending trend as a sign of improving financial discipline.
Edge case	Skipped if fewer than 3 months of data.

Rule SC-06: No Transaction Data Neutral Baseline

Attribute	Value
Name	SC-06 No Transaction Data Neutral Baseline
Input	List<Transaction> txs (empty or single month only)
Logic	If fewer than 2 distinct calendar months of transactions exist: set spendingConsistencyScore to 60. No deductions from SC-01 through SC-05 apply.

Insight Templates

The engine evaluates all 12 templates and selects the 3–5 highest-priority triggered insights to surface. Templates that do not trigger (their condition is false) are skipped. All amounts use `NumberFormat.currency(locale: "en_AU", symbol: "$").format(amount)`. All dates use `DateFormat("d MMM").format(date)`.

Template IT-01: Over-Budget Alert (severity: alert, category: budget)

Condition: at least one budget where `spentAmount > monthlyLimit`.

Text template (single worst offender by overspend percentage):

"Heads up — your [category] budget is over. You've spent \$[spentAmount] against a \$[monthlyLimit] limit this month. Time to pump the brakes!"

Template IT-02: Approaching Budget Warning (severity: warning, category: budget)

Condition: at least one budget where `85% <= spentAmount/monthlyLimit < 100%` and the month is not yet over.

Text template:

"Your [category] spend is at [utilisation%]% of your budget with [daysLeft] days left this month. You've got \$[remaining] to work with — make it count."

Template IT-03: Overdue Bill Alert (severity: alert, category: bills)

Condition: any bill where `isPaid == false` and `dueDate < DateTime.now()`.

Text template (single most overdue bill):

"[billName] was due on [dueDate] and hasn't been marked as paid yet. That's [daysOverdue] days overdue — sorting this out will help your Bill Reliability score."

Template IT-04: Upcoming Bill Warning (severity: warning, category: bills)

Condition: any bill where isPaid == false and dueDate is between tomorrow and 5 days from now.

Text template:

"[billName] is coming up on [dueDate] — that's \$[amount] due in [daysUntilDue] days. You're on the notification list, but it's good to be ready!"

Template IT-05: Spending Spike Warning (severity: warning, category: spending)

Condition: SC-01 fired with a 30% or greater spike.

Text template:

"Your spending this month is [spikePercent%] higher than last month (\$[currentTotal] vs \$[previousTotal]). Worth taking a look at what's driving the jump."

Template IT-06: Category Drift Warning (severity: warning, category: spending)

Condition: SC-02 fired — at least one category increased more than 50% month-over-month.

Text template (single worst-drift category):

"Your [category] spending jumped [changePercent%] compared to last month (\$[previousAmount] -> \$[currentAmount]). Is that expected, or worth reviewing?"

Template IT-07: Positive Trend Celebration (severity: info, category: spending)

Condition: SC-05 fired — spending decreased for 2 consecutive months.

Text template:

"You've been spending less for two months running — great discipline! Your total this month is \$[currentTotal], down from \$[previousTotal] last month."

Template IT-08: On-Time Payment Streak Celebration (severity: info, category: bills)

Condition: BR-03 fired with a streak of 3 or more.

Text template:

"Nice work — you've paid your last [streakCount] bills on time. Your Bill Reliability score is reflecting that consistency. Keep it up!"

Template IT-09: Consistent Under-Budget Celebration (severity: info, category: budget)

Condition: BA-04 fired — at least one category consistently under budget for 3+ months.

Text template:

"You've stayed under your [category] budget for [monthCount] months in a row. That kind of consistency is exactly what builds long-term financial health."

Template IT-10: Unbudgeted Spending Nudge (severity: warning, category: budget)

Condition: BA-05 fired — unbudgeted spending exceeds 20% of total spending.

Text template:

"About [unbudgetedPercent%] of your spending this month isn't covered by a budget. Adding budgets for [topUnbudgetedCategory] would give you a clearer picture of where your money goes."

Template IT-11: Single-Category Concentration Warning (severity: warning, category: spending)

Condition: SC-04 fired — one category exceeds 60% of total spending.

Text template:

"[topCategory] is taking up [concentrationPercent%] of your total spending this month (\$[categoryAmount] of \$[totalAmount]). Is that intentional? Spreading spend across categories usually improves your Consistency score."

Template IT-12: General Health Encouragement (severity: info, category: general)

Condition: Always fires. This is the fallback insight that guarantees a minimum of 1 insight always renders.

Text template — selects variant based on overallScore:

- Score 80–100: "Your finances are looking great this month! A score of [score] puts you in great shape. Keep your current habits going."
- Score 60–79: "You're in good shape with a score of [score]. Staying on top of your budgets and bills is what keeps this number climbing."
- Score 40–59: "Your score is [score] — there's room to grow. Focus on the warnings above and you'll see this number move up quickly."
- Score 0–39: "Your score is [score] right now. Don't stress — every score is a starting point. Tackling your most urgent alerts above is the fastest path forward."

3.5 Riverpod Provider Design

financialIntelligenceProvider

Type: AsyncNotifierProvider<FinancialIntelligenceNotifier, FinancialIntelligenceResult>

Watches: transactionListProvider, budgetListProvider, billReminderListProvider

Logic: Uses ref.watch on all three providers. If any returns AsyncLoading, returns AsyncLoading. If any returns AsyncError, returns AsyncError. When all are AsyncData, calls FinancialIntelligenceService().analyse(txs, budgets, bills) synchronously and returns the result as AsyncData.

Refresh: Automatically rebuilds whenever any of the three source providers change — no manual invalidation needed.

Error boundary: Wraps the analyse() call in try/catch. If an unexpected exception occurs, returns AsyncError with a user-friendly message rather than propagating the crash.

3.6 API Endpoint Specifications

This feature introduces no new HTTP API endpoints. All data access is via the existing solidpod readPod and writePod functions, mediated through PodService. The two new BillReminder fields (notificationDate, paidDate) are written via the existing PodService.updateBillReminder() call, no new PodService methods are required.

3.7 Authentication and Authorization

No changes. The Financial Intelligence Layer operates entirely within an authenticated session. The financialIntelligenceProvider depends on providers that already require authentication. If the session expires, the existing session-expiry handling in the app redirects to login — the intelligence provider will be in an error state and the Dashboard will show its standard error UI.

3.8 Key Technical Decisions and Trade-offs

Decision: Rule-based engine over on-device SLM

Context: Intelligence features require data analysis. Options range from on-device ML models to simple heuristics.

Options considered:

- On-device SLM (e.g. Phi-3 via llama.cpp Flutter bindings): genuinely intelligent but requires 1–4GB model bundled in app, platform-specific build complexity consuming most of the 20-hour sprint budget.
- Third-party inference API (Groq, Anthropic): capable and fast but violates SOLID data sovereignty, user financial data would leave the device.
- Rule-based engine in pure Dart: deterministic, fully sovereign, testable, zero dependencies, buildable in ~8 hours.

Chosen: Rule-based engine in pure Dart.

Rationale: Data sovereignty is the non-negotiable architectural constraint of Project Sanctum. Any solution that sends user financial data to an external service contradicts the project's core thesis. The rule-based engine delivers meaningful, friendly, specific insights without compromising this principle. For a 20-hour sprint, this is also the only option that leaves sufficient time for UI and testing.

Decision: Derived Riverpod provider over a new PodService method

Context: The intelligence result could be computed in PodService (alongside data loading) or in a separate Riverpod layer.

Options considered:

- In PodService: couples data access with analysis logic, violates single-responsibility principle, makes unit testing harder.
- Derived Riverpod provider watching existing providers: clean separation, automatically reactive to data changes, testable in isolation.

Chosen: Derived Riverpod provider (financialIntelligenceProvider).

Rationale: Keeps PodService as the single place responsible for Pod I/O and Turtle serialization. The intelligence layer is pure computation over already-loaded data. This matches the existing architectural pattern and requires zero new Pod reads.

Decision: Omit null Turtle predicates rather than serializing null literals

Context: The two new BillReminder fields are nullable. Their Turtle representation must be defined.

Options considered:

- Serialize nulls as empty string literals: `fin:notificationDate ""`. Simple but semantically wrong — an empty string is not an absent value in RDF.
- Serialize nulls as `rdf:nil`: technically correct but adds namespace complexity.
- Omit the predicate entirely when null: cleanest RDF — absence of a predicate is the canonical representation of "not applicable".

Chosen: Omit predicate when null.

Rationale: Absence of a predicate in RDF means the relationship does not exist — which is semantically correct for "this bill has not yet been notified". Existing Pod files without the predicate parse cleanly with null as default. This is also the pattern already used by the healthpod reference implementation.

APPENDIX

A. Document Revision History

Version	Date	Author	Changes
1.0	April 2026	Cyrill Adrian Wicaksono	Initial draft, all three parts complete. Covers BillReminder extension, 18-rule engine, 12 insight templates, Riverpod design, and UI widget specs.

B. Sign-Offs

Role	Name	Signature	Date
Author / Developer	Cyrill Adrian Wicaksono		
ANU Supervisor			

C. Supporting Documents and Links

- Project Sanctum Sprint Planning: Notion (Sprint 1 through 6 full specification)
- Project Sanctum Progress Log: Notion (Sprint 1 and 2 completion records)
- healthpod reference implementation: github.com/anusii/healthpod
- solidpod Flutter package: pub.dev/packages/solidpod
- SOLID specification: solidproject.org
- rdflib Dart package: pub.dev/packages/rdflib
- fl_chart Flutter package: pub.dev/packages/fl_chart